2. Arbeiten mit Linux

2.1. Dateien

Eine der fundamentalen Unix-Philosophien lautet: Alles ist eine Datei. Und das gilt eben auch für Geräte, also für die Tastatur oder eine USB-Schnittstelle. Die Konsequenz: Für einen Anwender reicht es aus, Dateien lesen und schreiben zu können. Schreibende und lesende Programme können auch direkt verbunden werden, z. Bsp. über sogenante Pipes (es gibt benamte und unbenamte Pipes, darauf soll aber hier nicht eingegangen werden).

Linux unterschiedet mehrere Dateitypen, die über Attribute festgelegt werden:

- Normale Dateien (ordinary files)
- Verzeichnisse (Ordner, directories)
- Gerätedateien (Character- und Blockdevices)
- Pipes
- Links (symbolic links, hard links)

Gerätedateien sind an dem Buchstaben `b` (Blockdevice) bzw. `c` (Characterdevice) zu erkennen, in der folgende Liste gehören die ersten 4 Einträge dazu.

```
brw-rw----  1 root disk      1,   8 26. Jan 12:17 ram8
brw-rw----  1 root disk      1,   9 26. Jan 12:17 ram9
crw-rw-rw-  1 root root      1,   8 26. Jan 12:17 random
crw-rw-r--+ 1 root netdev   10, 242 26. Jan 12:17 rfkill
lrwxrwxrwx  1 root root           5 26. Jan 12:17 serial0 -> ttyS0
lrwxrwxrwx  1 root root           7 26. Jan 12:17 serial1 -> ttyAMA0
drwxrwxrwt  2 root root          40 13. Sep 03:58 shm
drwxr-xr-x  3 root root         220 26. Jan 12:17 snd
```

Wie zu erkennen ist, gehören zu einer Gerätedatei zwei Nummern, die so genannte Major- und die Minornumber. Sie dient zur kernelinternen Kennzeichnung der dazugehörigen Treiber. Bei regulären Dateien stände an dieser Stelle die Dateigröße.

In Linux-Systemen gibt es nur einen Verzeichnisbaum, also keine Laufwerke. Filesysteme auf USB-Sticks oder Festplatten werden über Verzeichnisse eingehängt. Auf Desktop-Systemen sind unter `/media`

liegende Unterverzeichnisse für das Einbinden von „Hot-Plug-Devices“ üblich, z. Bsp. `/media/user/usbstick`. Das Verzeichnis sollte leer sein, da sonst die vorhandenen Dateien nicht mehr sichtbar sind. Das Einhängen eines Filesystems wird als Mounten bezeichnet, das Aushängen als Umounten.

Ein klassisches Linux-Userland hat die in der folgenden Tabelle aufgezeigte Verzeichnisstruktur. Die Verzeichnisse `/proc`, `/sys`, `/tmp` und manchmal auch `/var` sind virtuelle Verzeichnisse. Ihre Ordner und Dateien werden dynamisch generiert, die Daten liegen real im Hauptspeicher und nicht auf einem Hintergrundspeicher (Festplatte).

Verzeichnis	**Bedeutung**
/	Root-Verzeichnis
/bin	Elementare Systemprogramme
/sbin	Kommandos für die Systemverwaltung
/boot	Boot-Dateien (Kernel etc.)
/dev	Gerätedateien
/etc	Konfigurationsdateien
/home	Persönliche Daten der Nutzer
/root	Heimverzeichnis des Superusers (root)
/usr	Anwendungsprogramme
/lib	Bibliotheken
/proc	Informationen über das laufende System
/sys	Informationen über Geräte und Treiber
/tmp	Temporäre Dateien (typischerweise nach Reset gelöscht)
/var	Logdateien, Spooldateien, Lockdateien

Konfigurationen zu Programmen, wie beispielsweise einem Webserver oder auch die Internetnetkonfiguration überhaupt befinden sich typischerweise in Textdateien, die mit einem Editor bearbeitet werden können. Der Editor ist auch eines der wichtigsten Werkzeuge für den Entwickler.

Auf einem Linux-System mit grafischer Oberfläche (Ubuntu) gibt es z. Bsp. die Editoren `gedit` oder `kate`, die viele nützliche Funktionen mitbringen und eine fast selbserklärende Bedienung aufweisen. Da es aber auf einem eingebetteten System typischerweise keine grafische

Oberfläche gibt, muss man auf Editoren zurückgreifen, die nur ein Textterminal voraussetzen.

Viele sind vom dem „Urgestein“ `vi` abgeleitet, allerdings ist dieser Editor nicht leicht zu erlernen – so werden z. Bsp. auch die Cursorbewegungen über spezielle Tastenkombinationen gesteuert. Wer den Einarbeitungsaufwand scheut, sollte sich den Editor `nano` ansehen, der ebenfalls direkt auf einem Terminal verwendet werden kann. Er hat den Vorteil, dass benötigte Kommandos im unteren Bildschirmbereich eingeblendet sind (das Hütchen ^ steht für die Taste <Strg>).

Man kann sich natürlich auch per Netwerk mit dem Embedded System verbinden, über das Netzwerk die zu editierende Datei auf das Hostsystem holen, sie dort editieren und sie dann wieder zurückschreiben. Wir werden reichlich von dieser Möglichkeit gebrauch machen: Auf dem Embedded System läuft dann *open-ssh*; das den Dienst *scp* (secure copy protocol) zur Verfügung stellt. Ist der Hostrechner ein Windows System, kann dort das Tool *winscp* eingesetzt werden.

2.2. Arbeiten mit der Shell

Bei einer Shell handelt es sich um ein Programm, das Eingaben von der Tastatur entgegennimmt, interpretiert und als Kommandos ausführt. Wir werden Shell-Kommandos verwenden, um Verzeichnisse anzulegen, im Verzeichnisbaum zu navigieren, Dateien zu erzeugen und anzusehen usw.

Die Standard-Shell bei Linux heisst `bash` (borne again shell). Die Shell wird automatisch gestartet, sobald man sich an einer Textkonsole anmeldet oder auf einer grafischen Oberfläche (z. Bsp. KDE) ein Konsolenfenster öffnet.

Die folgende Tabelle zeigt einige nützliche Kommandos:

Kommando	**Funktion**
`ls`	Inhalt von Verzeichnissen listen. Bsp.: `ls -l /etc`
`cp`	Dateien kopieren
`mv`	Dateien umbennnen (verschieben)

`echo`	Text ausgeben
`mkdir`	Verzeichnis anlegen
`kill`	Task beenden. Bsp.: `kill -9 1554`
`ps`	Taskliste ausgeben. Bsp: `ps -ef`
`man`	Anzeigen der Manualpage. Bsp.: `man ps`
`ln`	Symbolischer Link anlegen. Bsp.: `ln -s <datei> <link>`
`pwd`	Name des Verzeichnisses ausgeben
`rm rmdir`	Datei bzw. Verzeichnis löschen. Bsp.: `rm -r /home/projekt1`
`chmod`	Zugriffsrechte ändern
`chown`	Besitzverhältnisse ändern
`cat`	Inhalte einer Datei ausgeben
`dd`	Dateien blockweise auslesen bzw. abspeichern
`tar`	Dateien archivieren. Bsp.: `tar -cvzf proj1.tgz /home/proj1`
`install`	Dateine kopieren und mit Rechten ausstatten
`reboot`	Rechner herunterfahren und neu booten
`tail`	Die letzten Zeilen einer Datei ausgeben
`find`	Das Dateisystem nach bestimmten Kriterien durchsuchen
`grep`	Datei durchsuchen
`mount, umount`	Dateisystem ein- bzw. aushängen
`insmod, modporbe`	Laden von Kernelmodulen
`lsmod, rmmod`	Listen und Löschen von Kernelmodulen
`mkfs.ext4, mkfs.vfat`	Dateisysteme anlegen (formatieren)
`mknod`	Gerätedatei erzeugen
`ifconfig`	Konfiguration des Netzwerk-Interfaces
`route, ip`	Konfiguration der Routing-Tabelle
`ftp`	FTP Client (File Transfer Protocol)
`ping`	Erreichbarkeit prüfen
`ssh, scp`	Remote Shell (secure shell) aktiviern, Dateien sicher kopieren

Mehrere Shell-Kommandos können in einer Befehlsdatei, dem Shell-Skript, zusammengefasst werden. Dabei sind auch die aus Programmiersprachen bekannte Konstrukte wie Schleifen und Verzweigungen möglich, worauf hier aber nicht eingegangen werden soll. Erwähnt sei aber noch die mögliche Umleitung des Eingabe-Streams `stdin` und der Ausgabe-Streams `stdout` und `stderr`. Auch die Möglichkeit, einen Task in den Hintergrund zu schieben oder zwei Tasks über eine unbenamte Pipe zu verbinden, soll hier erwähnt werden. Beispiele dazu sind in der folgenden Tabelle zu finden:

`>`	Das „>“-Zeichen wird anderen Kommandos angehängt, um die Standardausgabe (stdout) des Kommandos umzulenken, z. Bsp. in eine Datei.	`echo 1234 > /dev/treiber`
`2>`	Hier wird der Error-Stream (stderr) umgelenkt	`make > make.log 2> make.err`
`<`	Das „<“-Zeichen, dient dazu, die Eingabe nicht von der Tastatur, sondern aus einer Datei zu nehmen.	`./a.out < data.in`
`&`	Das „&“-Zeichen dient dazu, den Task in den Hintergrund zu schieben – die Shell steht dann sofort für neue Kommandos bereit	`./a.out > d.out &`
`|`	Das „|“-Zeichen verbindet den Output des ersten Kommandos mit dem Input des zweiten	`mount | grep sda`

Wir verweisen auf das Demoprogramm **`ap0/get_time.c`**

Mit `cat` lassen sich auch auf Dateien der virtuellen Dateisysteme anzeigen. Dadurch kann man wichtige Informationen über das System abrufen. Bsp.:

```
cat /proc/version
cat /proc/interrupts
cat /proc/cpuinfo
cat /proc/iomem
```

2.3. Die GNU-Compiler-Collection

GCC ist die Abkürzung für GNU Compiler Collection, ein weit verbreiteter C-Compiler für Linux. Der GCC wird auch zur Kompilierung des Linux-Kernels eingesetzt. Der Aufruf erfolgt über die Kommandozeile, der Aufruf der einzelnen Tools wird über Optionen gesteuert. Die wichtigsten Kommandozeilenoptionen sind in der folgenden Tabelle aufgelistet:

option	description
gcc -c	compile source files to object files without linking
gcc -D*name*[=*value*]	define a preprocessor macro
gcc -fPIC	generate position independent code for shared libraries
gcc -glevel	generate debug information to be used by GDB
gcc -Idir	add include directory of header files
gcc -llib	link with library file
gcc -Ldir	look in directory for library files
gcc -o output file	write build output to output file
gcc -Olevel	optimize for code size and execution time
gcc -shared	generate shared object file for shared library
gcc -U*name*	undefine a preprocessor macro
gcc -w	disable all warning messages
gcc -Wall	enable all warning messages
gcc -Wextra	enable extra warning messages

Die folgende Tabelle zeigt einige typische Anwendungsfälle des *gcc*-Kommandos:

Compile *file1.c* and *file2.c* and link to output file *execfile*	**gcc file1.c file2.c -o execfile**
Run output file *execfile*	**./execfile**
Compile *file1.c* and *file2.c* without linking	**gcc -c file1.c file2.c**
Compile *myfile.c* with debug information and link to output file *execfile*	**gcc -g myfile.c -o execfile**
Compile *myfile.c* with warning messages enabled and link to output file *execfile*	**gcc -Wall myfile.c -o execfile**
Compile *myfile.c* with and link with static library *libmath.a* located in */user/local/math* to output file *execfile*	**gcc -static myfile.c -L/user/local/math -lmath -o execfile**
Compile *myfile.c* with optimization and link to output file *execfile*	**gcc -O myfile.c -o execfile**

Der ISO-C99-Standard ermöglicht noch zwei im Folgenden sehr nützliche Möglichkeiten:

Makros mit variabler Argumentliste

Ab dem C99-Standard kann ein Makro mit variablen Argumenten definiert und aufgerufen werden.
Die Syntax ist vergleich mit der von Funktionen mit variabler Anzahl von Argumenten: Bei der Deklaration steht eine Folge von 3 Punkten (…). Beim Aufruf eines Makros fasst der Präprozessor alle optionalen Argumente zu einem Argument zusammen. Hierzu wurde der Bezeichner __VA_ARGS__ eingeführt, der im Ersatztext die zusammengefassten optionalen Argumente enthält. Allerdings darf der Bezeichner __VA_ARGS__ nur im Ersatztext der Makrodefinition verwendet werden. __VA_ARGS__ kann dort wie fast ein gewöhnlicher Parameter verwendet werden. Der Unterschied ist, dass der Parameter durch sämtliche Argumente ersetzt wird und nicht nur durch ein Argument.

Hier folgt ein einfaches Beispiel dafür, wie Sie ein Makro erstellen können, das eine variable Anzahl von Argumenten verwendet:

```
#include <stdio.h>

#ifndef DEB_LEV
#define DEB_LEV 0
#endif

#define PRINT(level,...) if(level<DEB_LEV) printf (__VA_ARGS__);

int main(void) {
     int a=0x123, b=0xabc;
     PRINT(0,"wichtige Meldung!\n");
     PRINT(1,"weniger wichtig: %08x %08x\n",a,b);
}

(übersetzt z. Bsp. mit gcc print_with_macro.c -DDEB_LEV=2)
```

Designated Initialization

Designated Initialization ist eine Erweiterung bei der Initialisierung von aggregierten Datentypen (Felder, Strukturen, Unions). Die Designated Initialization ermöglicht es, einzelne Elemente bei der Initialisierung auszuwählen. Dieses Konzept wurde mit dem ISO C99 Standard eingeführt. In GNU-C ist das auch schon im C90 Modus möglich.

Standard C90 sieht vor, dass die Elemente eines Aggregates in der festgelegten Reihenfolge initialisiert werden. In C99 können die Elemente eine beliebige Reihenfolge aufweisen, die Liste kann auch unvollständig sei. Die Elemente werden über Spezifikation der Feldindizes oder der Elementnamen zugeordnet.

Beispiel Felder:

```
int a[6] = { [4] = 29, [2] = 15 };
```

ist gleichbedeutend mit

```
int a[6] = { 0, 0, 15, 0, 29, 0 };
```

Beispiel Strukturen:

Deklaration der Struktur:

```
struct point { int x, y; };
```

Initialisierung klassisch:

```
struct point p = { xvalue, yvalue };
```

Initialisierung mit Designated Initializers:

```
struct point p = { .y = yvalue, .x = xvalue };
```

Weiteres Beispiel:

```
struct bla
{
  int a;
  int b;
  int c;
};

// klassische Initialisierung

struct bla eins = { 1, 2, 3 };

// mit "designated initializers"

struct bla zwei = { .b = 2, .a = 1, .c = 3 };
```

Bei der klassischen Initialisierung müssen die Initialisierer in der korrekten Reihenfolge angegeben werden, wenn nur ein Element der Struktur (mit einem von 0 abweichenden Wert) initialisiert werden soll, dann müssen alle Elemente der Struktur, die in der Deklaration vorher stehen, auch initialisiert werden.

Will man im obigen Beispiel nur das Strukturelement *c* initialisieren, müssen der Reihenfolge wegen zwangsweise auch *a* und *b* initialisiert werden.

Bei "designated initializers" ist die Reihenfolge beliebig, und so ist auch eine alleinige Initialisierung von *c* möglich; *a* und *b* werden dann mit dem Default-Wert (0) initialisiert.

Typischer späterer Anwendungsfall:

```
static struct file_operations fops = { /* definiert in
                                         linux/fs.h */
   .owner = THIS_MODULE,
   .read = device_read, /* Device File lesen */
   .write = device_write, /* schreiben */
   .open = device_open, /* Device File öffnen */
   .release = device_close, /*Device File schliessen (close)*/
};
```

2.4. Die Make-Utility

Die Make-Utility (*make*) ist ein Interpretierer für Makefiles. Ein Makefile entspricht einem Shellscript mit Regieanweisungen, die *make* dazu befähigen, nur die gerade erforderlichen Zeilen auszuführen.

Die Hauptanwendung ist natürlich das Kompilieren und Linken von C-Programmen. Aber mit *make* kann man auch andere Aufgaben organisieren. Die folgende Einführung löst sich daher zunächst von der Aufgabe der Programmerstellung.

Ein Makefile enthält im wesentlichen

- **Definitionen** von Variablen und Funktionen,
  z.B. `myshell=bash`
- **Kommentare**
  z.B. `# converting postscript to pdf`
- **Includes**
  z.B. `-include Makefile.local`
- **Regeln**
  z.B. `%.pdf: %.ps; -@ps2pdf $<`

Regeln

Eine Regel ist wie folgt aufgebaut:

```
Target [ weitere Targets] :[:] [ Vorbedingungen] [; Kommandos]
[  <tab>  Kommandos ]
[  <tab>  Kommandos ]
      ...
```

Die in eckigen Klammern dargestellten Teile sind optional.

In Abhänigkeit von „Target“ und „Vorbedingungen“ wird entschieden, ob und in welcher Reihenfolge die spezifizierten Kommandos ausgeführt werden. Das ist im Folgenden noch zu erläutern. Das „Target“ wird auch als abhängiger Teil der Bedingung bezeichet, die „Vorbedingungen“ als unabhängiger Teil.

Regeln *mit* Kommandos nennt man **explizit**, Regeln *ohne* Kommandos **implizit**. Wir beginnen mit den *expliziten* Regeln.

Wichtig ist das Tabulatorzeichen (<tab>) vor den Kommandos ab der zweiten Zeile.

Mehrere Kommandos in einer Zeile kann man mit Semikolon trennen; überlange Zeilen sollte man aufteilen und mit Backslash verbinden.

Die im Folgenden betrachteten Beispiele sind im Unterverzeichnis `make` in `ap0` zu finden.

Wir betrachten als erstes Beispiel ein Makefile, das zwei Regeln für die Targets *hello* und *diskfree* definiert; den Targets wird jeweils explizit ein kurzes Kommando zugeordnet:

```
hello:
        @echo hello \
        world

diskfree:; df -h /
```

Der Klammeraffe (`@`) vor dem `echo` verhindert übrigens die Ausgabe des auszuführenden Befehls.

Wir können diese Kommandos gezielt aufrufen:

```
$ make hello
hello world
$ make diskfree
df -h /
Filesystem            Size  Used Avail Use% Mounted on
/dev/hda6              43G   37G  3.6G  92% /
```

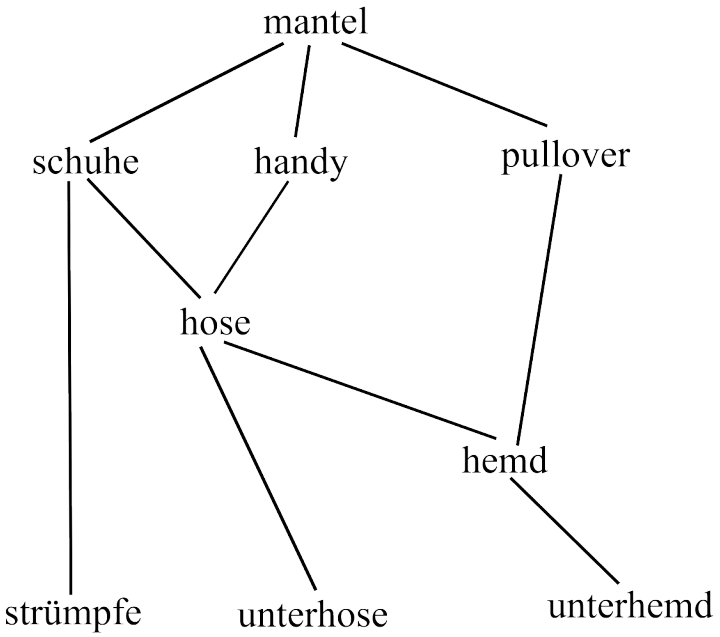
Zwei Feststellungen wollen wir noch festhalten:

- Wenn man beim `make`-Aufruf gar kein Target angibt, führt `make` einfach die erste Regel aus.
- Wenn man beim `make`-Aufruf mehrere Targets angibt, führt `make` diese alle in der Reihenfolge aus.

```
$ make
hello world
```

```
$ make diskfree hello
df -h /
Filesystem            Size  Used Avail Use% Mounted on
/dev/hda6              43G   37G  3.6G  92% /
hello world
```

Wir wollen uns jetzt der Frage zuwenden, wie Target und Vorbedingungen ausgewertet werden. Dazu diene das Alltagsbeispiel des morgendlichen Anziehens, das in einer bestimmten Reihenfolge von statten geht. Man kann die Abhängigkeiten wie folgt in einem Graphen veranschaulichen:



Aus dem Graphen ist zu ersehen, dass vor dem Anziehen der Schuhe beispielsweise Strümpfe und Hose bereits angezogen sein müssen: *Die Schuhe erfordern Strümpfe und Hose.*

Das notieren wir wie folgt in einem Makefile:

```
schuhe: struempfe hose
        @echo schuhe anziehen
```

Der Target-Name ist in der **automatischen Variablen** `$@` enthalten; unter Verwendung dieser automatischen Variablen können wir dann schreiben:

```
schuhe: struempfe hose
        @echo $@ anziehen
```

Setzt man dieses Verfahren für den gesamten Baum fort, so entsteht folgendes Makefile:

```
.PHONY: mantel schuhe handy pullover struempfe\
        hose hemd unterhose unterhemd

# Target    Vorbedingung           Befehl
# ------------------------------------------------
mantel:     schuhe handy pullover; @echo $@ anziehen
schuhe:     struempfe hose;        @echo $@ anziehen
handy:      hose;                  @echo $@ anziehen
pullover:   hemd;                  @echo $@ anziehen
struempfe:  ;                      @echo $@ anziehen
```

```
hose:       unterhose hemd;         @echo $@ anziehen
hemd:       unterhemd;              @echo $@ anziehen
unterhose:  ;                       @echo $@ anziehen
unterhemd:  ;                       @echo $@ anziehen
```

Die .PHONY-Anweisung bewirkt übrigens, dass die Targets nichts mit möglicherweise existierenden Dateien gleichen Namens zu tun haben. Darauf kommen wir noch zurück.

Beim Aufrufen von `make` verwenden wir noch das Unix-Proramm `nl`, verbunden über eine Pipe. Dieses Unix-Programm nummeriert nur die von `make` ausgegebenen Zeilen.

Wir erhalten das folgende Ergebnis:

```
$ make mantel | nl
     1  struempfe anziehen
     2  unterhose anziehen
     3  unterhemd anziehen
     4  hemd anziehen
     5  hose anziehen
     6  schuhe anziehen
     7  handy anziehen
     8  pullover anziehen
     9  mantel anziehen
```

Will man nur den Pullover anziehen und alles, was dazu erforderlich ist:

```
$ make pullover | nl
     1  unterhemd anziehen
     2  hemd anziehen
     3  pullover anziehen
```

Jetzt ist der Bezug der Targets auf vorhandene Dateien zu besprechen. Die .PHONY-Anweisung hat oben klar gemacht, dass das Target nichts mit einer vorhandenen Datei zu tun. Das ist aber in den meisten Anwendungsfällen sehr wohl der Fall (und auch der Default).

Wenn man in unserem Beispiel nach dem Anlegen der Grundausstattung erst einmal einen Kaffee trinken will, bevor der den Mantel angezogen wird, heisst das, dass *make* zunächst mit den Targets „hemd, hose, struempfe“ gestartet wird und nach dem „Kaffee trinken“ mit dem Target „mantel“. Bei diesem zweiten Start sollen

aber die ersten Schritte (Grundausstattung) nicht noch einmal ausgeführt werden.

Hierzu wird nach Abarbeitung eines Targets jeweils eine Datei angelegt, die genauso heisst wie das Target. Die .PHONY Zeile kommt weg, dann geht *make* beim Vorhandensein der Dateien davon aus, dass das Target bereits erfüllt ist (genau genommen vergleicht *make* das Dateidatum des Targets mit dem der Vorbedingung, darauf kommen wir später zurück).

Die Lösung sieht wie folgt aus:

```
# Target    Vorbedingung           Befehl
# ----------------------------------------------------------
mantel:     schuhe handy pullover; @echo $@ anziehen; touch $@
schuhe:     struempfe hose;        @echo $@ anziehen; touch $@
handy:      hose;                  @echo $@ anziehen; touch $@
pullover:   hemd;                  @echo $@ anziehen; touch $@
struempfe:  ;                      @echo $@ anziehen; touch $@
hose:       unterhose hemd;        @echo $@ anziehen; touch $@
hemd:       unterhemd;             @echo $@ anziehen; touch $@
unterhose:  ;                      @echo $@ anziehen; touch $@
unterhemd:  ;                      @echo $@ anziehen; touch $@
```

Sie funktioniert wie folgt:

```
$ make hemd hose struempfe | nl
     1  unterhemd anziehen
     2  hemd anziehen
     3  unterhose anziehen
     4  hose anziehen
     5  struempfe anziehen
$ ls
hose       unterhemd   hemd   struempfe  unterhose
$ # trinke kaffee
$ make mantel | nl
     1  schuhe anziehen
     2  handy anziehen
     3  pullover anziehen
     4  mantel anziehen
$ ls
hemd    mantel    schuhe     unterhemd   handy
hose    pullover  struempfe  unterhose
```

Man kann jede dieser Regeln in zwei Teile teilen:

- Eine *implizite* Regel erklärt die Abhängigkeiten und
- eine *explizite* Regel die Kommandos.

Alle Regeln im letzten Beispiel führten die selben Kommandos aus und unterschieden sich nur in den Vorbedingungen. Für die Regeln ohne Vorbedingungen (z.B. `struempfe`) benötigt man keine implizite Regel. Wegen der identischen Kommandos kann man die expliziten Regeln zusammenfassen. So entsteht folgendes, schon viel übersichtlichere Makefile:

```
# Eine explizite Regel definiert die Kommandos
mantel schuhe handy pullover struempfe hose\
hemd unterhose unterhemd: ;  @echo $@ anziehen; touch $@

# Implizite Regeln klären die Vorbedingungen
mantel:     schuhe handy pullover
schuhe:     struempfe hose
handy:      hose
pullover:   hemd
hose:       unterhose hemd
hemd:       unterhemd
```

Um die Sache noch übersichtlicher zu gestalten, kann man auch eigene Variablen (und Funktionen, darauf wird aber nicht eingegangen) definieren. Die automatische Variable **`$@`** haben wir bereits kennengelernt.

Eigene Variablen werden mit **`=`** oder **`:=`** deklariert, je nachdem ob die ggf. darin enthaltenen weiteren Variablen oder Funktionen bei **Verwendung** oder **Definition** der Variable ausgewertet werden sollen. Daneben gibt es noch die Zuweisung mit **`?=`** und **`+=`**. Im Folgenden werden diese Möglichkeiten kurz erläutert:

Simple assignment **`:=`**

Eine einfache Zuweisung wird nur einmal evaluiert.
Wenn `make` z. Bsp. Auf die Definition:
`CC :=$(GCC) $(FLAGS)`
trifft, wird diese sofort ausgewertet, z. Bsp. zu `gcc -W.`

Immer wenn danach `$(CC)` auftaucht, wird dies durch `gcc -W` ersetzt, auch wenn die Variablen `CC` bzw. `FLAGS` inzwischen neu definiert wurden.

Recursive assignment =

Eine rekursive Zuweisung wird jedes mal evaluiert, wenn die Variable auftaucht. Im obigen Beispiel würden damit zwischenzeitliche Änderung wie z. Bsp. `GCC=c++` relevant.

Conditional assignment ?=

Bedingte Zuweisungen werden nur ausgeführt, wenn die Variable noch undefiniert ist.

Appending +=

Der Operator += bewirkt ein Anhängen der neuen Definiton an die alte; z. Bsp. führt `CC = gcc` und ein anschließendes `CC += -W` dazu, dass `CC` den Wert `gcc -W` besitzt.

Wie aus diesen Beispielen hervorgeht, erhält man den in einer Variable `myvar` gespeicherten Text mit `$(myvar)`. Eine darin abgespeicherte Funktion könnte mit `$(call myvar)` ausgeführt werden.

Wir definieren nun eine Variable `kleidungsstuecke`, um unser Makefile noch übersichtlicher zu gestalten. Gleichzeitig wird auch ein Target `clean` definiert, um den Ausgangszustand wieder herstellen zu können. Das Minuszeichen (-) vor dem `rm` bewirkt übrigens, dass Fehler beim `rm` nicht zum Abbruch von `make` führen. Daher ist
`make clean mantel`
möglich, auch wenn vorher alles bereits „clean“ war.

```
# Definition einer Variable
kleidungsstuecke= mantel schuhe handy pullover\
        struempfe hose hemd unterhose unterhemd

# Kommando für alle Targets
$(kleidungsstuecke):; @echo $@ anziehen; touch $@

# Vorbedigungen für einige Kleidungsstücke
mantel:     schuhe handy pullover
schuhe:     struempfe hose
handy:      hose
pullover:   hemd
hose:       unterhose hemd
hemd:       unterhemd

# Zusaetzlicher Funktionsumfang
.PHONY: clean
clean:      ; @-rm $(kleidungsstuecke)
```

Größere Projekte (z.B. der Linux-Kernel) bringen oft so gewaltige Makefiles mit sich, dass man erst durch Aufteilen auf mehrere, kleinere Dateien den Überblick gewinnt („Teile und herrsche!).

Mit dem *include*-Befehl kann man weitere Makefiles an Ort und Stelle einfügen. Wenn man ihm ein Minuszeichen voranstellt, bricht *make* beim Fehlen dieses Files nicht ab.

```
# Krawatte anziehen
-include Makefile.krawatte
```

Insbesondere beim Arbeiten mit vielen Includes kann es vorkommen, dass eine Deklaration doppelt ausgeführt wird. Das führt zu einem Fehler, wie das folgende einfache Beispiel eines Makefiles zeigt, welches das Target `struempfe` verbotenerweise mehrfach definiert:

```
struempfe: ; @echo linken Strumpf anziehen
struempfe: ; @echo rechten Strumpf anziehen
```

`make` gibt daher eine Warnmeldung aus und führt nur das Kommando der letzten Deklaration aus:

```
$ make
Makefile:2: warning: overriding commands for target `struempfe'
Makefile:1: warning: ignoring old commands for target `struempfe'
rechten Strumpf anziehen
```

Wenn man, dass die Kommandos *beider* Deklarationen ausgeführt

werden, kann man statt dem einfachen Doppelpunkt einen zweifachen Doppelpunkt verwenden:

```
struempfe:: ; @echo linken Strumpf anziehen
struempfe:: ; @echo rechten Strumpf anziehen
```

Das Ergebnis:

```
$ make
linken Strumpf anziehen
rechten Strumpf anziehen
```

Das kann insbesondere bei der Arbeit mit *includes* von Nutzen sein.

Als letzes wollen wir noch den wichtigen Fall erörtern, wenn das Target aus und die Vorbedingungen aus Dateinamen bestehen (das ist der Default, der durch eine *Phony*-Deklaration überschrieben werden kann): in diesem Fall wird das Kommando nicht ausgeführt, wenn das Target bereits besteht und neuer ist als alle Vorbedingungen.

Beispiel:

```
testprogramm:   testprogramm.o
      gcc –o testprogramm testprogramm.o
```

In diesem Fall überprüft `make`, ob die Datei `testprogramm` (Target, abhängige Seite) existiert und ob sie älter oder jünger ist als die Datei `testprogramm.o` (Vorbedingung, unabhängige Seite). Ist `testprogramm` älter oder existiert `testprogramm` nicht, wird der Ausführungsteil gestartet.

Bevor aber das geschieht, überprüft `make` wiederum, ob die unabhängige Seite „up to date“, d. h. ob eine Regel hierfür existiert und diese ausgeführt werden muss. Passend zum obigen Beispiel wäre z. Bsp.

```
testprogramm.o:   testprogramm.c
      gcc –c –o testprogramm.o testprogramm.c
```

Ist z. Bsp. `testprogramm.c` jünger als `testprogramm.o`, so wird zunächst der Ausführungsteil dieser Regel abgearbeitet.

Außer `$@` gibt es noch weitere automatische Variablen, die wichtigsten davon sind in der folgenden Tabelle angegeben:

`$@`	Der Name des Target.
`$%`	Der Member-Name, falls das Target sich in einem Archiv befindet.
`$<`	Der Name der ersten (oder einzigen) Vorbedingung.
`$?`	Leerzeichen-getrennte Liste all derer Dateien aus der Vorbedingung, die neuer als das Target sind.
`$^` `$+`	Leerzeichen-getrennte Liste aller Vorbedingungen. Bei `$^` sind zusätzlich die Doppelten entfernt.

Zum Schluss noch ein Beispiel für ein einfaches Makefile:

```
CFLAGS=-g -O –Wall

all: treibertest.out

treibertest.o: treibertest.c
    $(CC) $(CFLAGS) -c treibertest.c

treibertest.out:
    $(CC) -o treibertest.out treibertest.o

devices:
    rm -f /dev/myfile
    mknod /dev/myfile c 240 0
    chmod a+rw /dev/myfile

clean:
    rm -f treibertest.out *.o *.bak

backup: clean
    cd ..
    tar -cvzf ~/backup/BUP_'date -u +%Y%j'.tgz .
```